Übung "Medienverwaltung": Aufgabe 1

Gegenstand der Übungsaufgaben im Fach Objektorientierte Programmierung ist es, den Bestand einer Bibliothek an Literatur und sonstigen Medien durch einen objektorientierten Entwurf zu repräsentieren.

Jedes Medium soll eine ganzzahlige Signatur, einen Titel und ein Attribut für den Leihstatus (präsent/entliehen) besitzen.

Implementieren Sie eine Klasse für diese Medien und schreiben Sie ein kleines kommando-gesteuertes Testprogramm dazu. Vorerst wird immer nur ein einzelnes Medium verwaltet. Das Programm erzeugt ein Medium beim Kommando "m" dynamisch und löscht es wieder nach Gebrauch (d.h. bevor beim nächsten Kommando "m" ein neues Medium angelegt wird).

Folgende Funktionen sollen sich mit diesem Medium in beliebiger Reihenfolge durchführen lassen:

Kommando	Aktion
"m"	Anlegen und Initialisieren des Mediums. Der Benutzer soll dazu mit der Meldung "Bitte Mediendaten eingeben: Signatur und Titel" zur Eingabe aufgefordert werden. Signatur- und Titeleingabe erfolgen jeweils in einer eigenen Zeile. Für den Titel reicht es aus, wenn er nur aus einem einzelnen Wort besteht.

"l"	Anzeige der Medieneigenschaften mit einer Spaltenüberschrift, etwa in folgender Form
	`Signatur  Typ  Titel  Status`
	`14432  Medium  Blechtrommel  vorh.`
	In dieser Aufgabe ist die Typbezeichnung immer "Medium".

"e"	Entleihen des zuletzt angelegten Mediums Es soll eine Plausibilitätskontrolle erfolgen, ob das Medium überhaupt präsent ist und ggf. eine Warnung ausgegeben werden.
"r"	Rückgabe des zuletzt angelegten Mediums Es soll eine Plausibilitätskontrolle erfolgen, ob das Medium überhaupt entliehen ist und ggf. eine Warnung ausgegeben werden.
"q"	Programmende

Hinweise:

1. Die Lösung ist so zu entwerfen, dass das Testprogramm keine detaillierten Kennnisse über die Attribute der Medienklasse besitzt. Wenn insbesondere das Medium weitere Eigenschaften erhält soll sich dies nur auf die Klassendefinition und nicht auf das übrige Programm auswirken.

2. Die Anwendung wird in weiteren Aufgaben noch ausgebaut. Insbesondere fügen wir eine Listenverwaltung für den Medienbestand und weitere Medientypen hinzu.

Übung "Medienverwaltung": Aufgabe 2

1.) Die in Aufgabe 1 begonnene Bibliothek verwaltet jetzt neben den allgemein gehaltenen Standard-Medien noch zwei konkrete Medienarten:
- Bücher, die neben Signatur, Titel und Leihstatus zusätzlich eine Seitenzahl besitzen,
- Videos, die zusätzlich eine Spieldauer besitzen (Wert in Minuten, nicht ganzzahlig).

Beim Anlegen eines neuen Buches bzw. Videos sollen die Benutzer mit den Texten

"Bitte Buchdaten eingeben: Signatur, Seitenzahl und Titel" bzw.
"Bitte Videodaten eingeben: Signatur, Spieldauer und Titel"

zur Eingabe der Stammdaten in genau dieser Reihenfolge aufgefordert werden.

Für Bücher und Videos soll wie bereits für die Medien eine Ausleihe/Rückgabe mit Plausibilitätskontrolle und eine einzeilige Anzeige der Eigenschaften möglich sein. Dabei wird die Spieldauer mit genau zwei Nachkommastellen dargestellt. Die Ausgabe muss mit festen Feldbreiten erfolgen, damit bei Auflistung mehrerer Medien eine Tabelle mit Spaltenstruktur entsteht. Der Titel soll aus mehreren Worten bestehen können und wie für Texte üblich linksbündig ausgerichtet werden. Zahlenwerte sind immer rechtsbündig platziert.

Implementieren Sie die Klassen für Bücher und Videos!

2.) Erweitern Sie das Testprogramm aus der ersten Aufgabe. Es sollen jetzt mehrere Medien, Bücher und Videos verwaltet werden können. Verwenden Sie dazu in dieser Aufgabe Arrays (Hinweise beachten!). Das Programm reagiert auf folgende Kommandos:

"m":	Hinzufügen und Initialisieren eines neuen Mediums
"b":	Hinzufügen und Initialisieren eines neuen Buchs
"v":	Hinzufügen und Initialisieren eines neuen Videos
"l":	Anzeige der Eigenschaften aller angelegten Objekte in tabellarischer Form mit einer kurzen Überschrift, etwa in folgender Form:

```
Medienbestand
  15457    Video   Avatar 3D      entl.   Dauer 110.50 min
   2466    Buch    Faust          vorh.   Seitenzahl 325
```

"e *Signatur*":	Entleihen des durch die Signatur bezeichneten Objekts
"r *Signatur*":	Rückgabe des durch die Signatur bezeichneten Objekts
"q":	Programmende und Löschen aller Objekte

Hinweise: (Nach wie vor ist Hinweis 1 von Übung 1 zu beachten!)
1. Das Einlesen der Daten soll für alle Objekte ausschließlich in geeigneten Konstruktoren (und nicht unter Zuhilfenahme von Initialisierungsmethoden) erfolgen.
2. Verwenden Sie mindestens eine Headerdatei für die Klassendefinitionen und implementieren Sie Hauptprogramm und Klassen in mindestens zwei getrennten Quelldateien!
3. Erzeugen Sie die Objekte wieder erst bei Bedarf und dann dynamisch!
4. Nach unserem bisherigen Kenntnisstand können Sie kein gemeinsames Array für alle Medientypen anlegen, vielmehr benötigt jeder Typ ein eigenes Array. Überlegen Sie sich warum! Später lernen wir eine Lösung kennen, die mit nur einem Array auskommt.
5. Achten Sie auf Arrayüberläufe!
6. Das Testprogramm braucht nicht alle möglichen Fehlbedienungen zu erkennen: Aber wenigstens soll exemplarisch die Signatureingabe beim Kommando "m" so stabil ausgeführt werden, dass Fehleingaben zu einer erneuten Eingabeaufforderung und nicht zum Absturz bzw. Endlosschleife führen.
7. Duplikate (weitere Medien mit gleicher Signatur) brauchen nicht erkannt zu werden.
8. Verwenden Sie die Klasse 'string' wo immer möglich!

Übung "Medienverwaltung": Aufgabe 3

Der Bestand der Bibliothek soll nun nicht mehr in einem Array, sondern in einer verketteten Liste gespeichert werden.

In dieser Aufgabe werden nur die Medien aus Aufgabe 1 verwaltet. Lassen Sie die Klassen für Videos und Bücher aber bestehen, weil wir diese in Aufgabe 4 wieder brauchen.

Die Liste muss als Klasse definiert werden und ihre Implementierung muss die folgenden Anforderungen erfüllen:

1. Die Listenklasse muss der Spezifikation einer Containerklasse aus dem separaten Umdruck genügen, d.h. die dort angegebenen Methoden besitzen.
2. Die bisherige Medienklasse darf gegenüber Aufgabe 2 nicht verändert werden, nur weil sie in eine übergeordnete Datenstruktur aufgenommen wird.
3. Die neue Listenklasse soll ohne Änderung ihrer Implementierung auch zur Verwaltung von Objekten beliebiger anderer Typen einsetzbar sein (z.B. für Zahlenwerte oder Zeichenketten). Zur Anpassung an einen anderen Typ müssen aber möglicherweise die Datentypen einiger Variablen geändert werden. Verwenden Sie eine typedef-Deklaration, damit diese Anpassungen nur das Ändern einer einzigen Zeile erfordern.

Mit dem Hauptprogramm soll sich der Bibliotheksbestand aufbauen und verwalten lassen. Die Kommandos und ihre Spezifikation sind dieselben wie in Aufgabe 2 mit folgenden Abweichungen:

"m"	Existiert bereits ein Medium mit derselben Signatur (Duplikat), wird die Aktion mit Fehlermeldung abgelehnt.
"b", "v":	Diese Kommandos werden vorübergehend deaktiviert.
"l":	Die Titel werden zwar in beliebiger Länge eingegeben und gespeichert. Bei der Ausgabe werden überlange Titel aber rechts abgeschnitten, sodass die Spaltenstruktur der Tabelle nicht zerstört wird.
"d *Signatur*":	Das durch die Signatur angegebene Medium wird aus der Liste entfernt und danach gelöscht.
"q":	Entfernen aller Medien aus der Liste und Löschen aller Medien, danach Programmende.

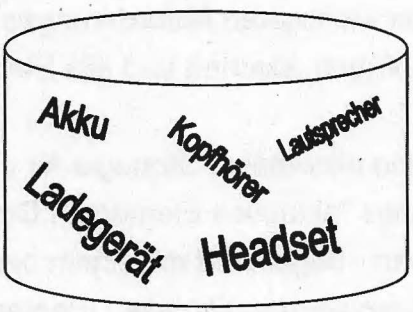
Hinweise:

- Wer nicht Angewandte Informatik studiert, braucht das Kommando "d" und das Löschen beim Kommando "q" nicht zu implementieren. Da bei den meisten Systemen (aber nicht bei allen!) die Laufzeitbibliothek des Compilers verbleibende dynamische Variablen beim Programmende automatisch löscht, kommt es zu keiner Beeinträchtigung des Systemverhaltens wenn das Löschen im Programm selbst unterbleibt.
- Damit Sie bei den Kommandos "d" und "q" das korrekte Löschen der dynamisch erzeugten Instanzen auf einfache Weise testen können, definieren Sie geeignete Destruktoren, die Kontrollmeldungen auf die Konsole schreiben! Die Meldungen sollen die Signatur des betroffenen Mediums enthalten.
- Für neue Anforderungen, die über die Listenverwaltung hinausgehen (z.B. Duplikate, Destruktor, lange Titel), darf die Medienklasse falls nötig durchaus verändert werden.
- Definieren Sie die Listenklasse in eigener Header- und Quelldatei!
- Vermeiden Sie öffentliche Elementvariablen!

Containerklassen

Vorbesprechung zu Übung 3 und Kapitel 13

Ein "**Container**" (manchmal auch "Collection" genannt) ist eine Datenstruktur, welche mehrere Elemente eines bestimmten Typs verwaltet.
Im Sinne der Objektorientierung wird ein solcher Container als Klasse realisiert, deren interne Repräsentation zunächst unerheblich ist.



Für die öffentliche Schnittstelle eines Containers gibt es mehrere Gestaltungsvarianten. Ein einfaches Containerdesign könnte zum Beispiel mit folgenden Methoden auskommen (wobei `T` der Typ der Containerelemente ist):

Container	
`+ Container()`	Konstruktor für leeren Container
`+ add(T* elem): bool`	Einfügen des Elements `elem` in den Container [1,3] (Returnwert: Fehlerkennung)
`+ remove(): bool`	Entfernen des aktuellen Elements aus Container [3] (Returnwert: Fehlerkennung)
`+ begin(): void`	Macht erstes Element zum aktuellen Element [2] Setzt Iterator auf erstes Element
`+ next(): void`	Macht nächstes Element zum aktuellen Element [2] Setzt Iterator auf nächstes Element
`+ getitem(): T*`	Liefert Zeiger auf aktuelles Element zurück [2]

[1] Die Einfügeposition ist von der Containerimplementierung abhängig.

[2] Wenn der Container leer bzw. dessen Ende erreicht, gibt es nach dem Aufruf von `begin()` bzw. `next()` kein aktuelles Element. Die Methode `getitem()` soll in diesem Fall den Nullzeiger zurückliefern. Dann sind anschließende Aufrufe von `next()` und `remove()` nicht erlaubt.

[3] Welches Element – wenn überhaupt eines - nach dieser Operation aktuell ist, ist von der Containerimplementierung abhängig und von ihr zu spezifizieren.

Die wichtigsten Anforderungen an einen Container sind, dass man ein Element einfügen, löschen und alle Elemente nacheinander auch wieder auslesen kann.

Eine allgemeine Strategie für den Zugriff auf Containerelemente ist die Verwendung eines "aktuellen Elements". Der Container muss dann Methoden bereitstellen, damit man - beginnend mit einem bestimmten ersten Element - alle Elemente nacheinander zum aktuellen Element machen kann ("den Container durchwandern" kann). In welcher Reihenfolge dies erfolgt, ist interne Angelegenheit des Containers. Üblicherweise werden diese Methoden in einer Schleife aufgerufen, mit der man dann "über die Containerelemente iterieren" kann.

Im objektorientierten Jargon verwendet man im Zusammenhang mit dem aktuellen Element gerne den Begriff des "**Iterators**" (manchmal auch "Enumerator" genannt). Ein Iterator ist konzeptionell ein **Zeiger auf das aktuelle Containerelement** [1]. Nur über einen solchen Iterator lassen sich dann einzelne Elemente adressieren, z.B. beim Zugriff oder beim Löschen.

Mit dem hier dargestellten Konzept der Containerklassen wird die interne Arbeitsweise der Datenstruktur vor dem Nutzer verborgen. Dies hat mehrere Vorteile:

- Einfachere Verwendung
- Fehlerhafte Manipulation der Datenstruktur wird weitgehend verhindert
- Verschiedene Implementierungen der Datenstruktur sind einfach austauschbar (Arrays, Stacks, Hashtabellen, Listen, Bäume).

Alle neueren Programmierumgebungen besitzen solche Container, wobei sich die Klassen und Methoden im Detail leicht unterscheiden (z.B. C++, Java, .NET).

[1] Er muss aber nicht notwendig als Zeigertyp der Programmiersprache realisiert sein. Für einen Container, der seine Elemente in einem Array speichert, könnte er z.B. auch durch einen ganzzahligen Index repräsentiert werden.

Übung "Medienverwaltung": Aufgabe 4

1.) Bei dieser Aufgabe soll es in der Bibliothek nur Bücher und Videos geben, die dann in den Container aufgenommen werden. Die ursprünglichen Medien ohne Zusatzeigenschaften entfallen. Daher sind die Kommandos "b" und "v" wieder zu reaktivieren und dafür das Kommando "m" stillzulegen.

2.) Beim Kommando "l" soll die Überschrift jetzt die Spaltennamen enthalten:

```
Signatur    Typ      Titel        Status    weitere Daten
  15457     Video    Avatar 3D    entl.     Dauer 110.50 min
   2466     Buch     Faust        vorh.     Seitenzahl 325
```

3.) Verwenden Sie für den Leihstatus statt einer booleschen Variablen einen Aufzählungstyp mit Wertevorrat "entliehen" und "praesent".

4.) Die Funktionen zum Entleihen und Rückgeben sollen bei Fehlschlag der Plausibilitätskontrolle nicht mehr wie bisher selbst eine Warnmeldung ausgeben, sondern statt dessen eine Exception mit dem selbstdefinierten Typ "StatusError" werfen. Beim Auslösen wird dieser Exception die ganzzahlige Signatur des betroffenen Mediums übergeben.

Das Hauptprogramm fängt diese Exception auf und sorgt nun selbst für die Fehlermeldung. Da das Hauptprogramm keine Kenntnis von den Details des Fehlers haben soll, beschafft es sich den passenden Fehlertext über eine Methode "message" der Exception-Klasse und gibt ihn aus. Die Methode "message" stellt den Fehlertext in einem C++-String <u>als Returnwert</u> bereit. Der Text hat die Form:

```
Fehler beim Leihen/Rueckgeben von Signatur 2665.
```

Nach dieser Meldung kann im Hauptprogramm das nächste Kommando eingegeben werden.

5.) Ergänzen Sie die Fehlerbehandlung von Punkt 4 so, dass bei jedem Leih-/Rückgabefehler <u>automatisch</u> ein Protokolleintrag in eine Log-Datei ERROR.LOG geschrieben wird. Der Eintrag lautet wie der Text in Punkt 4.
"Automatisch" heißt dabei:
- Das Hauptprogramm wird gegenüber Punkt 4 nicht verändert. Es ist also nicht an dem Log-Eintrag beteiligt. Selbst wenn das Hauptprogramm überhaupt kein Exception Handling enthielte, würde das Protokoll in die Datei geschrieben.
- Die Medienklassen sollen für den Log-Eintrag gegenüber Punkt 4 ebenfalls nicht verändert werden. Sie werfen also im Fehlerfall lediglich die Exception.

Daraus folgt, dass die Dateiausgabe von der Exception-Klasse abgewickelt werden muss.

Hinweise:
- Gemäß Hinweis 1 zu Aufgabe 1 sollen die konkreten Attribute (Elementvariablen) der Medienklassen nur in diesen Klassen selbst und nicht außerhalb bekannt sein. Dies gilt jetzt auch für die Spaltenüberschriften. Nur die Existenz einer Signatur darf außerhalb der Klasse vorausgesetzt werden, da diese das Identifikationsmerkmal für die Medien ist.
- Überlegen Sie sich, welche C++-Sprachelemente zur Realisierung von Punkt 1 und 2 besonders geeignet sind und setzen Sie diese auch ein!
- Ein Aufzählungstyp wird mit dem Schlüsselwort "enum" deklariert.